Subject State management Date

⇐ كيفية التحكم في البيانات المتغيرة أثناء تشغيل التطبيق ومتابعة التغيرات عشان يتم تحديث الـ UI على أساسه

* BloC

↳ قبل الـ BloC كنا بنكتب الكود كله في مكان واحد داخل في بعض (API Calls + UI + Business Logic) في نفس الـ screen
ده كان بيسبب مشاكل كتير

* Spaghetti Code ← كل الكود داخل في بعض
* مشكلة الـ Rebuild ← إنك تعمل setState عشان تغير حاجة بسيطة فا تبني الشاشة كلها ده شيء سيء

↳ الـ BloC يعتبر Architecture بيفصل كل حاجة عن التانية تماماً عشان المشروع يكون Scalable - Clean

* Bloc Architecture

1 Presentation layer 2 Business logic layer 3 Data layer

1 UI [Buttons - Text - widgets] أي حاجة اليوزر بيشوفها

* الوظيفة هي إنها تبعت Events لما اليوزر يعمل action وبتستقبل States عشان تحدث شكل الشاشة

2 BloC or Cubit → Business logic

* وظيفته هي إنه بيستقبل الـ Events من الـ UI ويروح للـ Data layer يطلب منه الداتا المناسبة ولما الداتا توصله على أساسها بيبعت State ويرجعها للـ UI
ex: loading State - Loaded State

③ Datalayer → مسئولة عن جلب البيانات سواء من API أو Db

① web services (API)

وظيفته إنه يكلم ال API ويجيب ال JSON زي ما هو ← Dio - http

② Models

ال classes اللي فيها بحول ال JSON لـ Data object وظيفة نقدر نتعامل معاهم بسهولة

③ Repository

دة اللي بيكلم web services وبعدين يجيب الداتا يبعتها لل Models وبعدين ياخد الداتا من ال Model يبعتها لل Bloc أو Cubit

* Data flow

UI (event) → BloC → Repository → web services → JSON → Repository → Model → Repository → BloC → UI

loading state

state

* BloC vs. Cubit

* ال Cubit هو نسخة مصغرة من ال BloC يعتمد على ال functions لتغيير الحالة ← ال UI بتستدعي function مباشرة من داخل ال Cubit وال function بتعمل حاجة معينة وتطلق حالة جديدة باستخدام emit

⇐ ex ← عند الضغط على Button لتحديث بيستدعي Cubit.fetch() فالتي ال Cubit بعمل emit(Data load)

* ال BloC هو النسخة الأكبر والأكثر تقدماً يعتمد على ال Events بدل من ال functions

← UI بيبعت Event لل BloC وال BloC يعمل اللي مفروض يعمله ثم يطلق حالة جديدة (emit)

ال Cubit مناسب للأبلكيشنز الصغيرة والمتوسطة وكود أقل

ال BloC مناسب لكل حاجة بس أحسن للأبلكيشنز الكبيرة لإنه كود أكبر

Mohamed & Amira Notebooks

* 4 Widgets to Connect UI ⇄ Bloc Cubit

① BlocProvider

هو اللي بيعمل Creat لل Bloc أو ال Cubit وبيوفرهم للـ Widgets اللي تحته في ال Widget tree

← بيشتغل بنظام ال Lazy loading مش هيعمل Create إلا لما ال UI يطلب الأول عشان يوفر في ال memory

② BlocBuilder

هو اللي بيبني ال UI أو بيعمل rebuild في كل مرة ال State تتغير فيها

خلي ال Bloc Builder أحسن في الأماكن اللي بيتعمله rebuild فقط مش فوق ال Scaffold عشان يحسن ال Performance

③ BlocListener

هو اللي بيـ Listen على ال State عشان ينفذ Action لمرة واحدة فقط

هو مش بيعمل rebuild لل UI خالص بنستخدمه مع الحاجات اللي محتاجة تصميم يتغير زي SnackBar - Toast - وقت Dialog أو BottomSheet على Navigation لشاشة تانية

④ BlocConsumer

عبارة عن ميكس بين الاثنين (Listener & Builder) في Widget واحدة

بنستخدمه لو إنت عندك شاشة محتاج لها الداتا تحمل تعرضها في List وفي نفس الوقت لو فيه error يطلع Dialog